

CASHLY EARN

Informe de Seguridad: Sesiones Autenticadas de Usuario

Implementado el 23 de agosto de 2026

Ø=Ý VULNERABILIDAD
CRÍTICA CORREGIDA

1. VULNERABILIDAD ORIGINAL — EMAIL SIN VERIFICACIÓN

¿Cuál era el problema?

Todos los endpoints de usuario regular aceptaban el email del cliente directamente en el body o como parámetro de URL, y ejecutaban operaciones de saldo o datos usando ese valor SIN verificar que la persona que hacía la petición fuera realmente dueña de ese email.

Ejemplo de ataque posible: Saldo de usuario no válido, acreditar saldo a otra cuenta, vaciarla, robar su historial o modificar su configuración simplemente enviando el email de otra persona.

2. SOLUCIÓN IMPLEMENTADA — TOKENS DE SESIÓN SERVER-SIDE

Arquitectura del sistema de tokens de usuario:

Se implementó un sistema de tokens de sesión para usuarios regulares idéntico al que ya existía para el admin (adminSessions), con las siguientes características:

- **Token generado:** 64 caracteres hexadecimales aleatorios (crypto.randomBytes(32))
- **Almacenamiento:** Solo en memoria del servidor (Map<token !' {email, expiresAt}>)
- **TTL:** 30 días — el token expira automáticamente
- **Limpieza:** Los tokens expirados se eliminan en cada issueUserToken()
- **Header HTTP:** x-user-token — enviado por el cliente en TODAS las peticiones
- **Persistencia:** No tiene

```
async function requireUser(req, res, next) {
  const token = req.headers["x-user-token"];
  if (!token) return res.status(401).json({ message: "No autorizado." });
  const user = await validateUserToken(token); // busca en Map servidor
  if (!user) return res.status(401).json({ message: "No autorizado." });
  req.userEmail = user.email; // email verificado — nunca del cliente
  next();
}
```

Middleware requireUser (servidor):

3. ENDPOINTS MODIFICADOS — RESUMEN COMPLETO

Autenticación — ahora emiten userToken en la respuesta:

POST /api/auth/login
POST /api/auth/signup
POST /api/auth/google

Emite userToken para usuario regular en respuesta JSON
Emite userToken al crear nueva cuenta
Emite userToken tras autenticar con Google

Nuevos endpoints de usuario (requieren x-user-token):

GET /api/user/balance
GET /api/user/history
GET /api/user/notifications
DELETE /api/user/notifications

Saldo — email del token, no del URL
Historial — email del token
Notificaciones — email del token
Borrar notificaciones — email del token

GET

[/api/user/settings](#)

Configuración — email del token

Endpoints existentes reforzados con requireUser:

POST	/api/user/earn
POST	/api/user/withdraw
POST	/api/user/notifications/read
PUT	/api/user/settings
PUT	/api/user/password
DELETE	/api/user/account
GET	/api/user/presence

Ganancias — email ignorado del body, viene del token

Retiro — email ignorado del body, viene del token

Marcar leídas — email del token

Guardar config — email del body ignorado, viene del token

Cambiar contraseña — email del body ignorado, viene del token

Borrar cuenta — email del body ignorado, viene del token

Beacon SSE — token en query param ?token= (SSE no admite headers)

Nuevo endpoint de cierre de sesión:

POST	/api/user/logout
------	------------------

Invalida el token del servidor inmediatamente

Endpoints /:email ahora requieren requireAdmin (solo panel admin):

GET	/api/user/balance/:email
GET	/api/user/history/:email
GET	/api/user/notifications/:email
DELETE	/api/user/notifications/:email
GET	/api/user/settings/:email

Solo admin — panel de usuarios

Solo admin — detalle de usuario

Solo admin

Solo admin

Solo admin — detalle de usuario

4. CAMBIOS EN EL FRONTEND

`lib/query-client.ts` — Nuevo `_userToken` module-level + `setUserToken()`. `apiRequest` y `getQueryFn` inyectan `x-user-token` automáticamente en TODAS las peticiones

`context/AuthContext.tsx` — Nuevo estado `userToken` + `USER_TOKEN_KEY` en `AsyncStorage`. `login/signup/google` guardan token del servidor. `logout` llama `/api/user/logout`

`context/BalanceContext.tsx` — `queryKey` cambiado a `["/api/user/balance"]` sin email. `earn` ya no envía email en body

`lib/notifications.ts` — Todas las funciones reescritas sin email en URL ni body. `markAllAsRead()` y `clearAllNotifications()` usan `apiRequest` directamente

`lib/settings.ts` — `getUserSettings()` y `saveUserSettings()` ya no reciben email como parámetro

`lib/history.ts` — `getEarnings()` y `getWithdrawals()` usan `/api/user/history` sin email en URL

`app/notifications.tsx` — `queryKey` y calls a lib sin email

`app/withdraw.tsx` — Body del retiro sin email. `queryKeys` sin email

`app/(tabs)/index.tsx` — `queryKey` de notificaciones sin email

`app/settings.tsx` — Calls a `getUserSettings()` y `saveUserSettings()` sin email

`app/history.tsx` — Calls a `getEarnings()` y `getWithdrawals()` sin email

5. CONFIRMACIÓN: NINGÚN ENDPOINT DEPENDE SOLO DEL EMAIL DEL CLIENTE

✓ VERIFICADO

Cada endpoint que modifica saldo, ganancias, retiros o datos sensibles de usuario:

1. Tiene el middleware `requireUser` aplicado
2. Obtiene el email del token verificado en el servidor (`req.userEmail`)
3. Ignora cualquier email que el cliente pudiera enviar en body o URL

4. Retorna 401 si el token no existe, expiró o es inválido

Los endpoints `/:email (admin)` ahora requieren `requireAdmin` — inaccesibles sin token de admin.

6. FLUJO COMPLETO DE AUTENTICACIÓN (NUEVO)

- \$`** **Usuario hace login/signup** !' Servidor verifica credenciales (bcrypt)
- \$a** **Servidor emite userToken** !' `randomBytes(32).toString("hex")` !' 30 días TTL
- \$b** **Cliente guarda token** !' `AsyncStorage` `@cashly_user_token` + estado React
- \$c** **query-client.ts lee _userToken** !' Se inyecta automáticamente en TODAS las peticiones
- \$d** **Servidor valida x-user-token** !' `requireUser` `middleware` !' `req.userEmail`
- \$e** **Operación ejecutada con email seguro** !' El servidor NUNCA confía en email del cliente
- \$f** **Logout: token invalidado** !' `userSessions.delete(token)` en servidor inmediatamente